

LLM Parameter Comparison Website

Idea

I want to build a website where a user can test the **same prompt with different combinations of LLM parameters** and compare the results.

The main purpose of the website is to help users understand:

For my particular task, which combination of LLM provider, model, effort, structured output, etc. gives me the best result?

Instead of manually testing different configurations one by one, the website should allow the user to select multiple parameters and automatically run all the resulting combinations.

User Flow

The website starts with a user entering a prompt.

Example:

Hello, What is Redis?

Before running the prompt, the user selects the LLM APIs/providers they want to test.

For example:

- ☒ OpenAI
- ☒ Gemini
- ☐ Anthropic

If the user selects OpenAI and Gemini, the next section should show options related to those two selected APIs.

Provider Selection

There are currently three possible providers:

OpenAI
Gemini
Anthropic

The user can select multiple providers.

For example:

Selected:

OpenAI
Gemini

Only the selected providers should continue into the next configuration level.

Model Selection

For every selected provider, the user should be able to select multiple models.

For example:

OpenAI

- ☒ Model 1
- ☒ Model 2

Gemini

- ☒ Model 3

The important concept is that **each selected model creates another branch in the configuration tree.**

Effort Selection

For every selected model, the user can select multiple effort levels.

For example:

OpenAI
Model 1

Effort:

- ☒ Low
- ☒ High

Another model can have its own selected efforts.

For example:

OpenAI

Model 2

Effort:

- ☒ Low
- ☒ Medium

The effort selections should therefore belong to the individual model/configuration rather than being treated as one global selection.

Structured Output Selection

For each selected configuration, there should be an option to enable or disable structured output.

For example:

OpenAI

→ Model 1

→ Low Effort

Structured Output:

- ☒ Enabled
- ☐ Disabled

If structured output is enabled, the user should then be able to select the desired structure/output option.

For the initial version, the options can include:

CSV

Pydantic

The idea is that the user should be able to compare:

Model + Effort + No Structured Output

against:

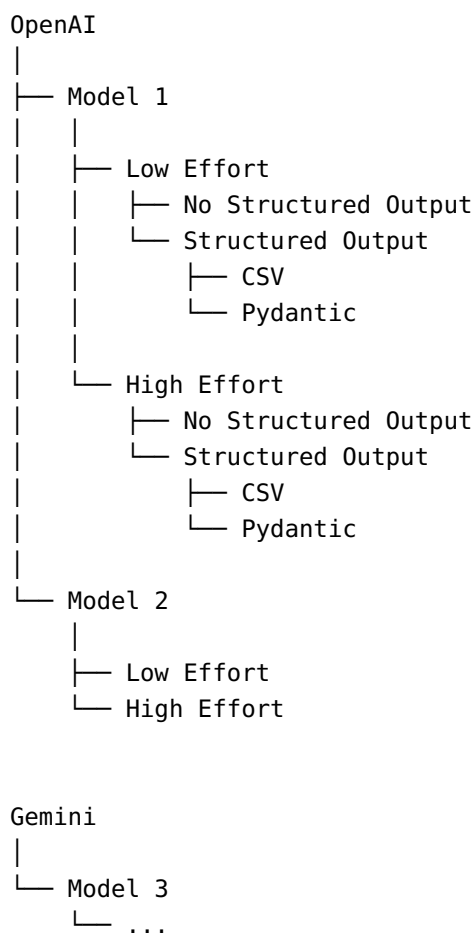
Model + Effort + Structured Output

and also compare different structured-output options.

The Tree / Combination Concept

The entire UI should behave like a tree.

For example:



Every final branch/leaf represents **one individual configuration to test**.

Example

Suppose the user selects:

Provider:
OpenAI

Model:
Model 1
Model 2

Effort:
Low
High

Structured Output:
Enabled
Disabled

The website should understand that the user wants to test the different combinations.

For example:

OpenAI
→ Model 1
→ Low Effort
→ No Structured Output

Then:

OpenAI
→ Model 1
→ Low Effort
→ Structured Output
→ Pydantic

Then:

OpenAI
→ Model 1
→ High Effort
→ No Structured Output

Then:

```
OpenAI
→ Model 2
→ Low Effort
→ Structured Output
→ CSV
```

And so on.

The system should run the prompt against each selected combination.

Experiment Result

The most important part of the website is the result of every individual combination.

For every configuration, the user should see:

Configuration

```
Provider:
OpenAI

Model:
Model 1

Effort:
Low

Structured Output:
Disabled
```

Response

The actual response generated for that configuration.

For example:

```
Redis is an in-memory data structure store...
```

Usage Information

The website should show information such as:

```
Input Tokens
Output Tokens
```

```
Total Tokens
Token Usage
```

and other relevant information returned by the API.

Other Information

The user should also be able to see what happened when that particular configuration was executed.

For example:

```
Provider: OpenAI
Model: Model 1
Effort: Low
Structured Output: No

Response: ...
Input Tokens: ...
Output Tokens: ...
Total Tokens: ...
```

The goal is to make the execution transparent.

The user should understand:

"This is exactly what happened when I selected this combination."

Multiple Results

If the user selects many combinations, the website should show the results for all of them.

For example:

```
-----
OpenAI → Model 1 → Low → No Structure
```

```
Response:
Redis is...
```

```
Input Tokens: 10
Output Tokens: 80
Total Tokens: 90
-----
```

```
OpenAI → Model 1 → High → No Structure
```

```
Response:
Redis is...
```

```
Input Tokens: 10
Output Tokens: 120
Total Tokens: 130
-----
```

```
OpenAI → Model 1 → Low → Pydantic
```

```
Response:
{
    ...
}
```

```
Input Tokens: 20
Output Tokens: 90
Total Tokens: 110
-----
```

The user can then compare the outcomes.

Main Purpose of the Website

The website is not simply a chatbot.

It is a **comparison and experimentation tool**.

The user should be able to answer questions such as:

- Which model gives the best response?
- Does higher effort improve the result?
- Does structured output change token usage?
- Which configuration uses fewer tokens?
- Which configuration gives a better response?
- Which configuration is more efficient?
- What happens when I change the parameters?

The website should make these differences visible by running the same prompt through different configurations.

Example Overall Flow

A user enters:

Hello, What is Redis?

Then selects:

Providers:

- ☒ OpenAI
- ☒ Gemini

Then:

OpenAI:

- ☒ Model 1
- ☒ Model 2

Gemini:

- ☒ Model 3

Then for each model:

Effort:

- ☒ Low
- ☒ High

Then:

Structured Output:

- ☒ No
- ☒ Yes

If Yes:

- ☒ CSV
- ☒ Pydantic

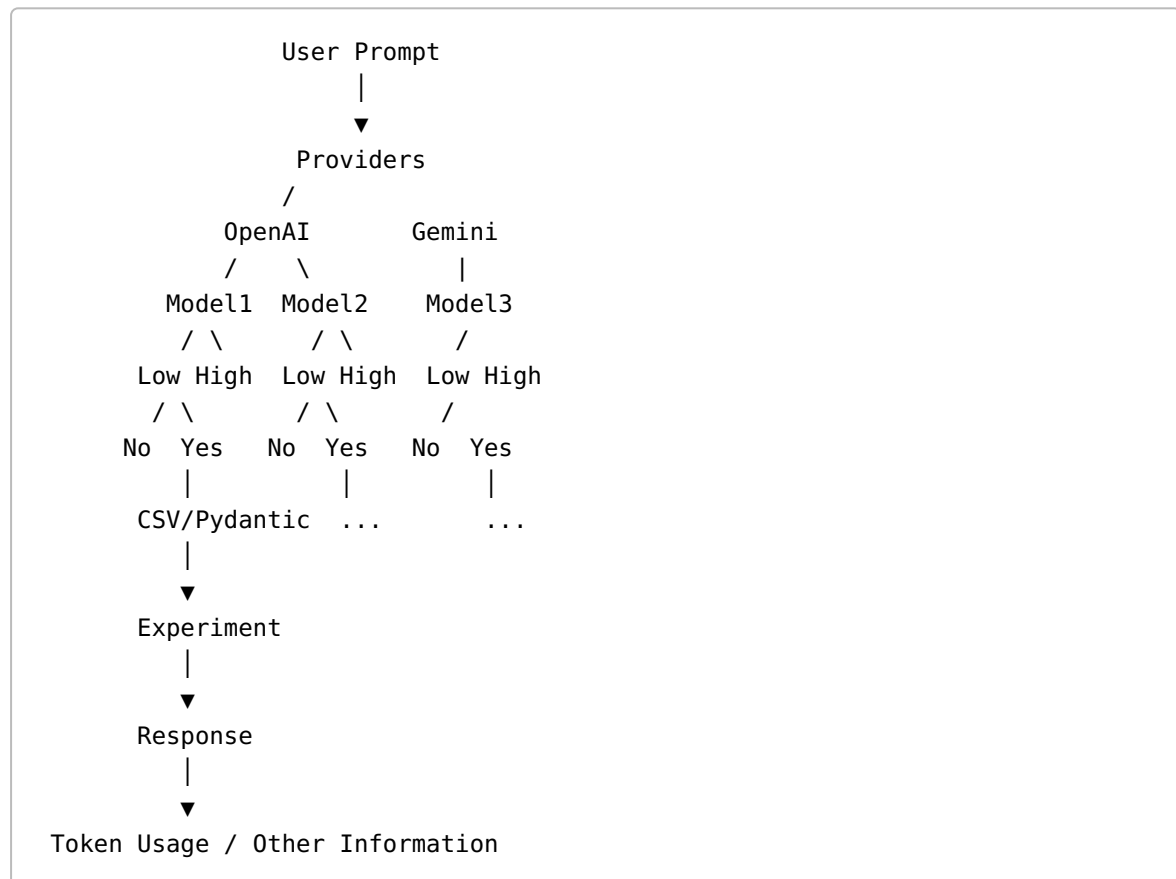
The website creates all the applicable combinations.

Each combination runs independently.

Then the website displays the response and information for every combination.

Core Concept

The entire application can be thought of as:



The **tree is created from the parameters selected by the user.**

Every different path through the tree represents a different experiment and can produce a different:

- Response
- Token usage
- Output
- Result
- Overall outcome

Final Goal

The final website should make it very easy for someone to experiment with different LLM configurations.

Instead of asking:

"Which model or parameter is better?"

the user can enter their **actual task/prompt**, select the configurations they want to test, run them together, and directly observe the differences.

The website should therefore help users discover:

Which configuration is most efficient and suitable for their specific task.

This is the complete core idea.

Do not change the core concept. Do not add unrelated features or turn it into a generic chatbot. The main idea is the **tree-based combination of LLM parameters and comparison of the resulting responses and usage for the same user prompt.**